

```

1  Library IEEE;
2  Use IEEE.std_logic_1164.all;
3  Use IEEE.numeric_std.all;
4
5  Entity ShiftUnit is
6      Generic ( N : natural := 64 );
7      Port (
8          A      : in  std_logic_vector(N-1 downto 0);
9          B      : in  std_logic_vector(N-1 downto 0);
10         ShiftFN : in  std_logic_vector(1 downto 0);
11         ExtWord : in  std_logic;
12         Y      : out std_logic_vector(N-1 downto 0)
13     );
14 End Entity ShiftUnit;
15
16 Architecture Dual2ChUni of ShiftUnit is
17
18     type Stage64 is array (0 to 6) of std_logic_vector(N-1 downto 0);
19     signal s64 : Stage64;
20     signal fill164 : std_logic;
21
22
23     type Stage32 is array (0 to 5) of std_logic_vector(N/2-1 downto 0);
24     signal s32 : Stage32;
25     signal fill132 : std_logic;
26
27     signal Y_32_extended : std_logic_vector(N-1 downto 0);
28     signal is_left : std_logic;
29 Begin
30
31     is_left <= '1' when ShiftFN = "01" else '0';
32
33
34     fill164 <= A(N-1) when ShiftFN = "11" else '0';
35     s64(0) <= A;
36
37     gen_64bit_chain: for i in 0 to 5 generate
38         gen_bits_64: for j in 0 to N-1 generate
39             process(s64, B, fill164, is_left)
40                 variable shift_amt : integer := 2**i;
41                 begin
42                     if is_left = '0' then
43                         if j + shift_amt <= N-1 then
44                             if B(i) = '1' then
45                                 s64(i+1)(j) <= s64(i)(j + shift_amt);
46                             else
47                                 s64(i+1)(j) <= s64(i)(j);
48                             end if;
49                         else
50                             if B(i) = '1' then
51                                 s64(i+1)(j) <= fill164;
52                             else
53                                 s64(i+1)(j) <= s64(i)(j);
54                             end if;
55                         end if;
56                     else
57                         if j - shift_amt >= 0 then
58                             if B(i) = '1' then
59                                 s64(i+1)(j) <= s64(i)(j - shift_amt);
60                             else
61                                 s64(i+1)(j) <= s64(i)(j);
62                             end if;
63                         else
64                             if B(i) = '1' then
65                                 s64(i+1)(j) <= '0';
66                             else
67                                 s64(i+1)(j) <= s64(i)(j);
68                             end if;
69                         end if;

```

```

70         end if;
71     end process;
72 end generate;
73 end generate;
74
75
76 fill132 <= A(N/2-1) when ShiftFN = "11" else '0';
77 s32(0) <= A(N/2-1 downto 0);
78
79 gen_32bit_chain: for i in 0 to 4 generate
80     gen_bits_32: for j in 0 to N/2-1 generate
81         process(s32, B, fill132, is_left)
82             variable shift_amt : integer := 2**i;
83         begin
84             if is_left = '0' then
85                 if j + shift_amt <= N/2-1 then
86                     if B(i) = '1' then
87                         s32(i+1)(j) <= s32(i)(j + shift_amt);
88                     else
89                         s32(i+1)(j) <= s32(i)(j);
90                     end if;
91                 else
92                     if B(i) = '1' then
93                         s32(i+1)(j) <= fill132;
94                     else
95                         s32(i+1)(j) <= s32(i)(j);
96                     end if;
97                 end if;
98             else
99                 if j - shift_amt >= 0 then
100                     if B(i) = '1' then
101                         s32(i+1)(j) <= s32(i)(j - shift_amt);
102                     else
103                         s32(i+1)(j) <= s32(i)(j);
104                     end if;
105                 else
106                     if B(i) = '1' then
107                         s32(i+1)(j) <= '0';
108                     else
109                         s32(i+1)(j) <= s32(i)(j);
110                     end if;
111                 end if;
112             end if;
113         end process;
114     end generate;
115 end generate;
116
117
118 Y_32_extended <= (N-1 downto N/2 => s32(5)(N/2-1)) & s32(5);
119
120
121 Y <= Y_32_extended when ExtWord = '1' else s64(6);
122
123 End Architecture Dual2ChUni;
124
125
126 Architecture Dual4ChUni of ShiftUnit is
127
128     -- 64-bit chain: 4 taps (input + 3 stage outputs)
129     type Stage64 is array (0 to 3) of std_logic_vector(63 downto 0);
130     signal s64      : Stage64;
131     signal fill164 : std_logic;
132
133     -- 32-bit chain: 4 taps (input + 3 stage outputs)
134     -- Stage 2 of 32-bit is effectively 2-channel (only 1 bit of shamt left)
135     type Stage32 is array (0 to 3) of std_logic_vector(31 downto 0);
136     signal s32    : Stage32;
137     signal fill132 : std_logic;
138

```

```

139     signal Y_32_ext : std_logic_vector(63 downto 0);
140     signal is_left  : std_logic;
141
142 Begin
143
144     is_left <= '1' when ShiftFN = "01" else '0';
145
146
147     fill164 <= A(63) when ShiftFN = "11" else '0';
148     s64(0) <= A;
149
150     gen_64_stages : for i in 0 to 2 generate
151         gen_64_bits : for j in 0 to 63 generate
152             process(s64, B, fill164, is_left)
153                 variable base : integer := 4**i;
154                 variable sel   : integer;
155                 variable src   : integer;
156             begin
157
158                 sel := to_integer(unsigned(B(2*i+1 downto 2*i)));
159
160                 if is_left = '0' then
161
162                     src := j + sel * base;
163                     if src <= 63 then
164                         s64(i+1)(j) <= s64(i)(src);
165                     else
166                         s64(i+1)(j) <= fill164;
167                     end if;
168                 else
169
170                     src := j - sel * base;
171                     if src >= 0 then
172                         s64(i+1)(j) <= s64(i)(src);
173                     else
174                         s64(i+1)(j) <= '0';
175                     end if;
176                 end if;
177             end process;
178         end generate;
179     end generate;
180
181
182     fill132 <= A(31) when ShiftFN = "11" else '0';
183     s32(0) <= A(31 downto 0);
184
185
186     gen_32_s0 : for j in 0 to 31 generate
187         process(s32, B, fill132, is_left)
188             variable sel : integer;
189             variable src : integer;
190         begin
191             sel := to_integer(unsigned(B(1 downto 0)));
192             if is_left = '0' then
193                 src := j + sel;
194                 if src <= 31 then
195                     s32(1)(j) <= s32(0)(src);
196                 else
197                     s32(1)(j) <= fill132;
198                 end if;
199             else
200                 src := j - sel;
201                 if src >= 0 then
202                     s32(1)(j) <= s32(0)(src);
203                 else
204                     s32(1)(j) <= '0';
205                 end if;
206             end if;
207         end process;

```

```

208     end generate;
209
210 gen_32_s1 : for j in 0 to 31 generate
211     process(s32, B, fill32, is_left)
212         variable sel : integer;
213         variable src : integer;
214     begin
215         sel := to_integer(unsigned(B(3 downto 2)));
216         if is_left = '0' then
217             src := j + sel * 4;
218             if src <= 31 then
219                 s32(2)(j) <= s32(1)(src);
220             else
221                 s32(2)(j) <= fill32;
222             end if;
223         else
224             src := j - sel * 4;
225             if src >= 0 then
226                 s32(2)(j) <= s32(1)(src);
227             else
228                 s32(2)(j) <= '0';
229             end if;
230         end if;
231     end process;
232 end generate;
233
234 gen_32_s2 : for j in 0 to 31 generate
235     process(s32, B, fill32, is_left)
236         variable src : integer;
237     begin
238         if is_left = '0' then
239             src := j + 16;
240             if B(4) = '1' then
241                 if src <= 31 then
242                     s32(3)(j) <= s32(2)(src);
243                 else
244                     s32(3)(j) <= fill32;
245                 end if;
246             else
247                 s32(3)(j) <= s32(2)(j);
248             end if;
249         else
250             src := j - 16;
251             if B(4) = '1' then
252                 if src >= 0 then
253                     s32(3)(j) <= s32(2)(src);
254                 else
255                     s32(3)(j) <= '0';
256                 end if;
257             else
258                 s32(3)(j) <= s32(2)(j);
259             end if;
260         end if;
261     end process;
262 end generate;
263
264
265 Y_32_ext <= (63 downto 32 => s32(3)(31)) & s32(3);
266
267
268 process(s64, Y_32_ext, A, ShiftFN, ExtWord)
269 begin
270     if ShiftFN = "00" then
271         Y <= A;
272     elsif ExtWord = '1' then
273         Y <= Y_32_ext;
274     else
275         Y <= s64(3);
276     end if;

```

```

277     end process;
278
279 End Architecture Dual4ChUni;
280
281
282 Architecture SingleZU2ChRev of ShiftUnit is
283     type StageArray is array (0 to 6) of std_logic_vector(N-1 downto 0);
284     signal stage      : StageArray;
285     signal A_in       : std_logic_vector(N-1 downto 0);
286     signal A_rev      : std_logic_vector(N-1 downto 0);
287     signal Y_rev      : std_logic_vector(N-1 downto 0);
288     signal Y_int       : std_logic_vector(N-1 downto 0);
289     signal shamt       : std_logic_vector(5 downto 0);
290     signal fill        : std_logic;
291     signal is_sll      : std_logic;
292 Begin
293     is_sll <= '1' when ShiftFN = "01" else '0';
294
295
296     process(A, ShiftFN, ExtWord)
297     begin
298         if ExtWord = '1' and ShiftFN = "11" then
299             A_in(N/2-1 downto 0) <= A(N/2-1 downto 0);
300             A_in(N-1 downto N/2) <= (others => A(N/2-1));
301         elsif ExtWord = '1' then
302             A_in(N/2-1 downto 0) <= A(N/2-1 downto 0);
303             A_in(N-1 downto N/2) <= (others => '0');
304         else
305             A_in <= A;
306         end if;
307     end process;
308
309     shamt(4 downto 0) <= B(4 downto 0);
310     shamt(5)          <= B(5) when ExtWord = '0' else '0';
311
312
313     fill <= A(N/2-1) when (ShiftFN = "11" and ExtWord = '1')
314         else A(N-1) when (ShiftFN = "11" and ExtWord = '0')
315         else '0';
316
317
318     gen_rev_in : for i in 0 to N-1 generate
319         A_rev(i) <= A_in(N-1-i) when is_sll = '1' else A_in(i);
320     end generate;
321
322     stage(0) <= A_rev;
323
324     gen_stages : for i in 0 to 5 generate
325         gen_bits : for j in 0 to N-1 generate
326             process(stage, shamt, fill)
327                 variable shift_amt : integer := 2**i;
328             begin
329                 if j + shift_amt <= N-1 then
330                     if shamt(i) = '1' then
331                         stage(i+1)(j) <= stage(i)(j + shift_amt);
332                     else
333                         stage(i+1)(j) <= stage(i)(j);
334                     end if;
335                 else
336                     if shamt(i) = '1' then
337                         stage(i+1)(j) <= fill;
338                     else
339                         stage(i+1)(j) <= stage(i)(j);
340                     end if;
341                 end if;
342             end process;
343         end generate;
344     end generate;
345

```

```

346
347     gen_rev_out : for i in 0 to N-1 generate
348         Y_rev(i) <= stage(6)(N-1-i) when is_sll = '1' else stage(6)(i);
349     end generate;
350
351     Y_int <= Y_rev;
352
353     -- Output
354     process(Y_int, A, ShiftFN, ExtWord)
355     begin
356         if ShiftFN = "00" then
357             Y <= A;
358         else
359             Y(N/2-1 downto 0) <= Y_int(N/2-1 downto 0);
360             if ExtWord = '1' then
361                 Y(N-1 downto N/2) <= (others => Y_int(N/2-1));
362             else
363                 Y(N-1 downto N/2) <= Y_int(N-1 downto N/2);
364             end if;
365         end if;
366     end process;
367
368 End Architecture SingleZU2ChRev;
369
370
371 Architecture SingleZU4ChRev of ShiftUnit is
372
373     type StageArray is array (0 to 3) of std_logic_vector(N-1 downto 0);
374     signal stage      : StageArray;
375
376     signal A_in       : std_logic_vector(N-1 downto 0);
377     signal A_rev      : std_logic_vector(N-1 downto 0);
378     signal Y_rev      : std_logic_vector(N-1 downto 0);
379     signal Y_int      : std_logic_vector(N-1 downto 0);
380
381     signal shamt      : std_logic_vector(5 downto 0);
382     signal fill       : std_logic;
383     signal is_sll     : std_logic;
384
385 Begin
386
387     is_sll <= '1' when ShiftFN = "01" else '0';
388
389     process(A, ShiftFN, ExtWord)
390     begin
391         if ExtWord = '1' and ShiftFN = "11" then
392             A_in(31 downto 0) <= A(31 downto 0);
393             A_in(63 downto 32) <= (others => A(31));
394         elsif ExtWord = '1' then
395             A_in(31 downto 0) <= A(31 downto 0);
396             A_in(63 downto 32) <= (others => '0');
397         else
398             A_in <= A;
399         end if;
400     end process;
401
402     shamt(4 downto 0) <= B(4 downto 0);
403     shamt(5) <= B(5) when ExtWord = '0' else '0';
404
405     fill <= A(31) when (ShiftFN = "11" and ExtWord = '1')
406         else A(63) when (ShiftFN = "11" and ExtWord = '0')
407         else '0';
408
409     gen_rev_in : for i in 0 to N-1 generate
410         A_rev(i) <= A_in(N-1-i) when is_sll = '1' else A_in(i);
411     end generate;
412
413     stage(0) <= A_rev;
414

```

```

415 gen_stages : for i in 0 to 2 generate
416     gen_bits : for j in 0 to N-1 generate
417         process(stage, shamt, fill)
418             variable base : integer := 4**i;
419             variable sel  : integer;
420             variable src  : integer;
421         begin
422             sel := to_integer(unsigned(shamt(2*i+1 downto 2*i)));
423             src := j + sel * base;
424             if src <= N-1 then
425                 stage(i+1)(j) <= stage(i)(src);
426             else
427                 stage(i+1)(j) <= fill;
428             end if;
429         end process;
430     end generate;
431 end generate;
432
433 gen_rev_out : for i in 0 to N-1 generate
434     Y_rev(i) <= stage(3)(N-1-i) when is_sll = '1' else stage(3)(i);
435 end generate;
436
437 Y_int <= Y_rev;
438
439 process(Y_int, A, ShiftFN, ExtWord)
440 begin
441     if ShiftFN = "00" then
442         Y <= A;
443     else
444         Y(31 downto 0) <= Y_int(31 downto 0);
445         if ExtWord = '1' then
446             Y(63 downto 32) <= (others => Y_int(31));
447         else
448             Y(63 downto 32) <= Y_int(63 downto 32);
449         end if;
450     end if;
451 end process;
452
453 End Architecture SingleZU4ChRev;
454
455 Architecture Dual8ChUni of ShiftUnit is
456
457     type Stage64 is array (0 to 2) of std_logic_vector(63 downto 0);
458     signal s64    : Stage64;
459     signal fill64 : std_logic;
460
461
462     type Stage32 is array (0 to 2) of std_logic_vector(31 downto 0);
463     signal s32    : Stage32;
464     signal fill32 : std_logic;
465
466     signal Y_32_ext : std_logic_vector(63 downto 0);
467     signal is_left  : std_logic;
468
469 Begin
470
471     is_left <= '1' when ShiftFN = "01" else '0';
472
473
474     fill64 <= A(63) when ShiftFN = "11" else '0';
475     s64(0) <= A;
476
477     gen_64_s0 : for j in 0 to 63 generate
478         process(s64, B, fill64, is_left)
479             variable sel : integer;
480             variable src : integer;
481         begin
482             sel := to_integer(unsigned(B(2 downto 0)));

```

```

484         if is_left = '0' then
485             src := j + sel;
486             if src <= 63 then s64(1)(j) <= s64(0)(src);
487             else s64(1)(j) <= fill164; end if;
488         else
489             src := j - sel;
490             if src >= 0 then s64(1)(j) <= s64(0)(src);
491             else s64(1)(j) <= '0'; end if;
492         end if;
493     end process;
494 end generate;
495
496 gen_64_s1 : for j in 0 to 63 generate
497     process(s64, B, fill164, is_left)
498         variable sel : integer;
499         variable src : integer;
500     begin
501         sel := to_integer(unsigned(B(5 downto 3)));
502         if is_left = '0' then
503             src := j + sel * 8;
504             if src <= 63 then s64(2)(j) <= s64(1)(src);
505             else s64(2)(j) <= fill164; end if;
506         else
507             src := j - sel * 8;
508             if src >= 0 then s64(2)(j) <= s64(1)(src);
509             else s64(2)(j) <= '0'; end if;
510         end if;
511     end process;
512 end generate;
513
514 fill132 <= A(31) when ShiftFN = "11" else '0';
515 s32(0) <= A(31 downto 0);
516
517
518 gen_32_s0 : for j in 0 to 31 generate
519     process(s32, B, fill132, is_left)
520         variable sel : integer;
521         variable src : integer;
522     begin
523         sel := to_integer(unsigned(B(2 downto 0)));
524         if is_left = '0' then
525             src := j + sel;
526             if src <= 31 then s32(1)(j) <= s32(0)(src);
527             else s32(1)(j) <= fill132; end if;
528         else
529             src := j - sel;
530             if src >= 0 then s32(1)(j) <= s32(0)(src);
531             else s32(1)(j) <= '0'; end if;
532         end if;
533     end process;
534 end generate;
535
536 gen_32_s1 : for j in 0 to 31 generate
537     process(s32, B, fill132, is_left)
538         variable sel : integer;
539         variable src : integer;
540     begin
541         sel := to_integer(unsigned(B(4 downto 3)));
542         if is_left = '0' then
543             src := j + sel * 8;
544             if src <= 31 then s32(2)(j) <= s32(1)(src);
545             else s32(2)(j) <= fill132; end if;
546         else
547             src := j - sel * 8;
548             if src >= 0 then s32(2)(j) <= s32(1)(src);
549             else s32(2)(j) <= '0'; end if;
550         end if;
551     end process;
552 end generate;

```



```

553
554 Y_32_ext <= (63 downto 32 => s32(2)(31)) & s32(2);
555
556
557 process(s64, Y_32_ext, A, ShiftFN, ExtWord)
558 begin
559     if ShiftFN = "00" then
560         Y <= A;
561     elsif ExtWord = '1' then
562         Y <= Y_32_ext;
563     else
564         Y <= s64(2);
565     end if;
566 end process;
567
568 End Architecture Dual8ChUni;
569
570
571 Architecture SingleZU8ChRev of ShiftUnit is
572
573     type StageArray is array (0 to 2) of std_logic_vector(N-1 downto 0);
574     signal stage      : StageArray;
575
576     signal A_in       : std_logic_vector(N-1 downto 0);
577     signal A_rev      : std_logic_vector(N-1 downto 0);
578     signal Y_rev      : std_logic_vector(N-1 downto 0);
579     signal Y_int       : std_logic_vector(N-1 downto 0);
580
581     signal shamt       : std_logic_vector(5 downto 0);
582     signal fill        : std_logic;
583     signal is_sll      : std_logic;
584
585 Begin
586
587     is_sll <= '1' when ShiftFN = "01" else '0';
588
589     process(A, ShiftFN, ExtWord)
590     begin
591         if ExtWord = '1' and ShiftFN = "11" then
592             A_in(31 downto 0) <= A(31 downto 0);
593             A_in(63 downto 32) <= (others => A(31));
594         elsif ExtWord = '1' then
595             A_in(31 downto 0) <= A(31 downto 0);
596             A_in(63 downto 32) <= (others => '0');
597         else
598             A_in <= A;
599         end if;
600     end process;
601
602     shamt(4 downto 0) <= B(4 downto 0);
603     shamt(5)          <= B(5) when ExtWord = '0' else '0';
604
605     fill <= A(31) when (ShiftFN = "11" and ExtWord = '1')
606            else A(63) when (ShiftFN = "11" and ExtWord = '0')
607            else '0';
608
609     gen_rev_in : for i in 0 to N-1 generate
610         A_rev(i) <= A_in(N-1-i) when is_sll = '1' else A_in(i);
611     end generate;
612
613     stage(0) <= A_rev;
614
615     gen_s0 : for j in 0 to N-1 generate
616         process(stage, shamt, fill)
617             variable sel : integer;
618             variable src : integer;
619         begin
620             sel := to_integer(unsigned(shamt(2 downto 0)));
621             src := j + sel;

```

```

622         if src <= N-1 then stage(1)(j) <= stage(0)(src);
623         else stage(1)(j) <= fill; end if;
624     end process;
625 end generate;
626
627
628 gen_sl : for j in 0 to N-1 generate
629     process(stage, shamt, fill)
630         variable sel : integer;
631         variable src : integer;
632     begin
633         sel := to_integer(unsigned(shamt(5 downto 3)));
634         src := j + sel * 8;
635         if src <= N-1 then stage(2)(j) <= stage(1)(src);
636         else stage(2)(j) <= fill; end if;
637     end process;
638 end generate;
639
640 gen_rev_out : for i in 0 to N-1 generate
641     Y_rev(i) <= stage(2)(N-1-i) when is_sll = '1' else stage(2)(i);
642 end generate;
643
644 Y_int <= Y_rev;
645
646
647 process(Y_int, A, ShiftFN, ExtWord)
648 begin
649     if ShiftFN = "00" then
650         Y <= A;
651     else
652         Y(31 downto 0) <= Y_int(31 downto 0);
653         if ExtWord = '1' then
654             Y(63 downto 32) <= (others => Y_int(31));
655         else
656             Y(63 downto 32) <= Y_int(63 downto 32);
657         end if;
658     end if;
659 end process;
660
661 End Architecture SingleZU8ChRev;
662
663
664 Architecture SingleSW2ChUni of ShiftUnit is
665
666     type StageArray is array (0 to 6) of std_logic_vector(N-1 downto 0);
667     signal stage : StageArray;
668
669     signal A_in : std_logic_vector(N-1 downto 0);
670     signal Y_int : std_logic_vector(N-1 downto 0);
671
672     signal shamt : std_logic_vector(5 downto 0);
673     signal fill : std_logic;
674     signal is_left : std_logic;
675
676 Begin
677
678     is_left <= '1' when ShiftFN = "01" else '0';
679
680
681 process(A, ShiftFN, ExtWord)
682 begin
683     if ExtWord = '1' and ShiftFN /= "01" then
684
685         A_in(63 downto 32) <= A(31 downto 0);
686         A_in(31 downto 0) <= (others => '0');
687     elsif ExtWord = '1' and ShiftFN = "01" then
688
689         A_in(31 downto 0) <= A(31 downto 0);
690         A_in(63 downto 32) <= (others => '0');

```

```

691     else
692         A_in <= A;
693     end if;
694 end process;
695
696
697 shamt(4 downto 0) <= B(4 downto 0);
698 shamt(5)          <= B(5) when ExtWord = '0' else '0';
699
700
701 fill <= A(31) when (ShiftFN = "11" and ExtWord = '1')
702     else A(63) when (ShiftFN = "11" and ExtWord = '0')
703     else '0';
704
705 stage(0) <= A_in;
706
707 gen_stages : for i in 0 to 5 generate
708     gen_bits : for j in 0 to N-1 generate
709         process(stage, shamt, fill, is_left)
710             variable shift_amt : integer := 2**i;
711             variable src       : integer;
712         begin
713             if is_left = '0' then
714                 src := j + shift_amt;
715                 if shamt(i) = '1' then
716                     if src <= N-1 then stage(i+1)(j) <= stage(i)(src);
717                     else                stage(i+1)(j) <= fill; end if;
718                 else
719                     stage(i+1)(j) <= stage(i)(j);
720                 end if;
721             else
722                 src := j - shift_amt;
723                 if shamt(i) = '1' then
724                     if src >= 0 then stage(i+1)(j) <= stage(i)(src);
725                     else            stage(i+1)(j) <= '0'; end if;
726                 else
727                     stage(i+1)(j) <= stage(i)(j);
728                 end if;
729             end if;
730         end process;
731     end generate;
732 end generate;
733
734 Y_int <= stage(6);
735
736
737 process(Y_int, A, ShiftFN, ExtWord)
738 begin
739     if ShiftFN = "00" then
740         Y <= A;
741     elsif ExtWord = '1' and ShiftFN /= "01" then
742
743         Y(31 downto 0) <= Y_int(63 downto 32);
744         Y(63 downto 32) <= (others => Y_int(63));
745     elsif ExtWord = '1' then
746
747         Y(31 downto 0) <= Y_int(31 downto 0);
748         Y(63 downto 32) <= (others => Y_int(31));
749     else
750         Y <= Y_int;
751     end if;
752 end process;
753 End Architecture SingleSW2ChUni;
754
755 Architecture SingleSW4ChUni of ShiftUnit is
756
757     type StageArray is array (0 to 3) of std_logic_vector(N-1 downto 0);
758     signal stage     : StageArray;
759

```

```

760     signal A_in      : std_logic_vector(N-1 downto 0);
761     signal Y_int     : std_logic_vector(N-1 downto 0);
762
763     signal shamt      : std_logic_vector(5 downto 0);
764     signal fill       : std_logic;
765     signal is_left    : std_logic;
766
767 Begin
768
769     is_left <= '1' when ShiftFN = "01" else '0';
770
771     process(A, ShiftFN, ExtWord)
772     begin
773         if ExtWord = '1' and ShiftFN /= "01" then
774             A_in(63 downto 32) <= A(31 downto 0);
775             A_in(31 downto 0)  <= (others => '0');
776         elsif ExtWord = '1' and ShiftFN = "01" then
777             A_in(31 downto 0)  <= A(31 downto 0);
778             A_in(63 downto 32) <= (others => '0');
779         else
780             A_in <= A;
781         end if;
782     end process;
783
784     shamt(4 downto 0) <= B(4 downto 0);
785     shamt(5)          <= B(5) when ExtWord = '0' else '0';
786
787     fill <= A(31) when (ShiftFN = "11" and ExtWord = '1')
788             else A(63) when (ShiftFN = "11" and ExtWord = '0')
789             else '0';
790
791     stage(0) <= A_in;
792
793
794     gen_stages : for i in 0 to 2 generate
795         gen_bits : for j in 0 to N-1 generate
796             process(stage, shamt, fill, is_left)
797                 variable base : integer := 4**i;
798                 variable sel  : integer;
799                 variable src  : integer;
800             begin
801                 sel := to_integer(unsigned(shamt(2*i+1 downto 2*i)));
802                 if is_left = '0' then
803                     src := j + sel * base;
804                     if src <= N-1 then stage(i+1)(j) <= stage(i)(src);
805                     else                stage(i+1)(j) <= fill; end if;
806                 else
807                     src := j - sel * base;
808                     if src >= 0 then stage(i+1)(j) <= stage(i)(src);
809                     else                stage(i+1)(j) <= '0'; end if;
810                 end if;
811             end process;
812         end generate;
813     end generate;
814
815     Y_int <= stage(3);
816
817
818     process(Y_int, A, ShiftFN, ExtWord)
819     begin
820         if ShiftFN = "00" then
821             Y <= A;
822         elsif ExtWord = '1' and ShiftFN /= "01" then
823
824             Y(31 downto 0) <= Y_int(63 downto 32);
825             Y(63 downto 32) <= (others => Y_int(63));
826         elsif ExtWord = '1' then
827
828             Y(31 downto 0) <= Y_int(31 downto 0);

```

```

829         Y(63 downto 32) <= (others => Y_int(31));
830     else
831         Y <= Y_int;
832     end if;
833 end process;
834
835 End Architecture SingleSW4ChUni;
836
837 Architecture Dual248ChUni of ShiftUnit is
838
839     type Stage64 is array (0 to 3) of std_logic_vector(63 downto 0);
840     signal s64      : Stage64;
841     signal fill64 : std_logic;
842
843     type Stage32 is array (0 to 3) of std_logic_vector(31 downto 0);
844     signal s32      : Stage32;
845     signal fill32 : std_logic;
846
847     signal Y_32ext : std_logic_vector(63 downto 0);
848     signal is_left : std_logic;
849
850 Begin
851
852     is_left <= '1' when ShiftFN = "01" else '0';
853     fill64  <= A(63) when ShiftFN = "11" else '0';
854     fill32  <= A(31) when ShiftFN = "11" else '0';
855     s64(0)  <= A;
856     s32(0)  <= A(31 downto 0);
857
858
859     gen_64_s0 : for j in 0 to 63 generate
860         process(s64, B, fill64, is_left)
861         begin
862             if is_left = '0' then
863                 if j + 1 <= 63 then
864                     if B(0) = '1' then s64(1)(j) <= s64(0)(j+1);
865                     else s64(1)(j) <= s64(0)(j); end if;
866                 else
867                     if B(0) = '1' then s64(1)(j) <= fill64;
868                     else s64(1)(j) <= s64(0)(j); end if;
869                 end if;
870             else
871                 if j - 1 >= 0 then
872                     if B(0) = '1' then s64(1)(j) <= s64(0)(j-1);
873                     else s64(1)(j) <= s64(0)(j); end if;
874                 else
875                     if B(0) = '1' then s64(1)(j) <= '0';
876                     else s64(1)(j) <= s64(0)(j); end if;
877                 end if;
878             end if;
879         end process;
880     end generate;
881
882
883     gen_64_s1 : for j in 0 to 63 generate
884         process(s64, B, fill64, is_left)
885             variable sel : integer;
886             variable src : integer;
887         begin
888             sel := to_integer(unsigned(B(2 downto 1)));
889             if is_left = '0' then
890                 src := j + sel * 2;
891                 if src <= 63 then s64(2)(j) <= s64(1)(src);
892                 else s64(2)(j) <= fill64; end if;
893             else
894                 src := j - sel * 2;
895                 if src >= 0 then s64(2)(j) <= s64(1)(src);
896                 else s64(2)(j) <= '0'; end if;
897             end if;

```

```

898     end process;
899 end generate;
900
901
902 gen_64_s2 : for j in 0 to 63 generate
903     process(s64, B, fill64, is_left)
904         variable sel : integer;
905         variable src : integer;
906     begin
907         sel := to_integer(unsigned(B(5 downto 3)));
908         if is_left = '0' then
909             src := j + sel * 8;
910             if src <= 63 then s64(3)(j) <= s64(2)(src);
911             else s64(3)(j) <= fill64; end if;
912         else
913             src := j - sel * 8;
914             if src >= 0 then s64(3)(j) <= s64(2)(src);
915             else s64(3)(j) <= '0'; end if;
916         end if;
917     end process;
918 end generate;
919
920
921 gen_32_s0 : for j in 0 to 31 generate
922     process(s32, B, fill32, is_left)
923     begin
924         if is_left = '0' then
925             if j + 1 <= 31 then
926                 if B(0) = '1' then s32(1)(j) <= s32(0)(j+1);
927                 else s32(1)(j) <= s32(0)(j); end if;
928             else
929                 if B(0) = '1' then s32(1)(j) <= fill32;
930                 else s32(1)(j) <= s32(0)(j); end if;
931             end if;
932         else
933             if j - 1 >= 0 then
934                 if B(0) = '1' then s32(1)(j) <= s32(0)(j-1);
935                 else s32(1)(j) <= s32(0)(j); end if;
936             else
937                 if B(0) = '1' then s32(1)(j) <= '0';
938                 else s32(1)(j) <= s32(0)(j); end if;
939             end if;
940         end if;
941     end process;
942 end generate;
943
944
945 gen_32_s1 : for j in 0 to 31 generate
946     process(s32, B, fill32, is_left)
947         variable sel : integer;
948         variable src : integer;
949     begin
950         sel := to_integer(unsigned(B(2 downto 1)));
951         if is_left = '0' then
952             src := j + sel * 2;
953             if src <= 31 then s32(2)(j) <= s32(1)(src);
954             else s32(2)(j) <= fill32; end if;
955         else
956             src := j - sel * 2;
957             if src >= 0 then s32(2)(j) <= s32(1)(src);
958             else s32(2)(j) <= '0'; end if;
959         end if;
960     end process;
961 end generate;
962
963
964 gen_32_s2 : for j in 0 to 31 generate
965     process(s32, B, fill32, is_left)
966         variable sel : integer;

```

```

967         variable src : integer;
968     begin
969         sel := to_integer(unsigned(B(4 downto 3)));
970         if is_left = '0' then
971             src := j + sel * 8;
972             if src <= 31 then s32(3)(j) <= s32(2)(src);
973             else s32(3)(j) <= fill132; end if;
974         else
975             src := j - sel * 8;
976             if src >= 0 then s32(3)(j) <= s32(2)(src);
977             else s32(3)(j) <= '0'; end if;
978         end if;
979     end process;
980 end generate;
981
982 Y_32ext <= (63 downto 32 => s32(3)(31)) & s32(3);
983
984
985 process(s64, Y_32ext, A, ShiftFN, ExtWord)
986 begin
987     if ShiftFN = "00" then
988         Y <= A;
989     elsif ExtWord = '1' then
990         Y <= Y_32ext;
991     else
992         Y <= s64(3);
993     end if;
994 end process;
995
996 End Architecture Dual248ChUni;
997
998 Architecture Dual824ChUni of ShiftUnit is
999
1000     type Stage64 is array (0 to 3) of std_logic_vector(63 downto 0);
1001     signal s64 : Stage64;
1002     signal fill164 : std_logic;
1003
1004     type Stage32 is array (0 to 3) of std_logic_vector(31 downto 0);
1005     signal s32 : Stage32;
1006     signal fill132 : std_logic;
1007
1008     signal Y_32ext : std_logic_vector(63 downto 0);
1009     signal is_left : std_logic;
1010
1011 Begin
1012
1013     is_left <= '1' when ShiftFN = "01" else '0';
1014     fill164 <= A(63) when ShiftFN = "11" else '0';
1015     fill132 <= A(31) when ShiftFN = "11" else '0';
1016     s64(0) <= A;
1017     s32(0) <= A(31 downto 0);
1018
1019
1020     gen_64_s0 : for j in 0 to 63 generate
1021         process(s64, B, fill164, is_left)
1022             variable sel : integer;
1023             variable src : integer;
1024         begin
1025             sel := to_integer(unsigned(B(2 downto 0)));
1026             if is_left = '0' then
1027                 src := j + sel;
1028                 if src <= 63 then s64(1)(j) <= s64(0)(src);
1029                 else s64(1)(j) <= fill164; end if;
1030             else
1031                 src := j - sel;
1032                 if src >= 0 then s64(1)(j) <= s64(0)(src);
1033                 else s64(1)(j) <= '0'; end if;
1034             end if;
1035         end process;

```

```

1036 end generate;
1037
1038
1039 gen_64_s1 : for j in 0 to 63 generate
1040     process(s64, B, fill164, is_left)
1041     begin
1042         if is_left = '0' then
1043             if j + 8 <= 63 then
1044                 if B(3) = '1' then s64(2)(j) <= s64(1)(j+8);
1045                 else s64(2)(j) <= s64(1)(j); end if;
1046             else
1047                 if B(3) = '1' then s64(2)(j) <= fill164;
1048                 else s64(2)(j) <= s64(1)(j); end if;
1049             end if;
1050         else
1051             if j - 8 >= 0 then
1052                 if B(3) = '1' then s64(2)(j) <= s64(1)(j-8);
1053                 else s64(2)(j) <= s64(1)(j); end if;
1054             else
1055                 if B(3) = '1' then s64(2)(j) <= '0';
1056                 else s64(2)(j) <= s64(1)(j); end if;
1057             end if;
1058         end if;
1059     end process;
1060 end generate;
1061
1062 gen_64_s2 : for j in 0 to 63 generate
1063     process(s64, B, fill164, is_left)
1064         variable sel : integer;
1065         variable src : integer;
1066     begin
1067         sel := to_integer(unsigned(B(5 downto 4)));
1068         if is_left = '0' then
1069             src := j + sel * 16;
1070             if src <= 63 then s64(3)(j) <= s64(2)(src);
1071             else s64(3)(j) <= fill164; end if;
1072         else
1073             src := j - sel * 16;
1074             if src >= 0 then s64(3)(j) <= s64(2)(src);
1075             else s64(3)(j) <= '0'; end if;
1076         end if;
1077     end process;
1078 end generate;
1079
1080
1081 gen_32_s0 : for j in 0 to 31 generate
1082     process(s32, B, fill132, is_left)
1083         variable sel : integer;
1084         variable src : integer;
1085     begin
1086         sel := to_integer(unsigned(B(2 downto 0)));
1087         if is_left = '0' then
1088             src := j + sel;
1089             if src <= 31 then s32(1)(j) <= s32(0)(src);
1090             else s32(1)(j) <= fill132; end if;
1091         else
1092             src := j - sel;
1093             if src >= 0 then s32(1)(j) <= s32(0)(src);
1094             else s32(1)(j) <= '0'; end if;
1095         end if;
1096     end process;
1097 end generate;
1098
1099
1100 gen_32_s1 : for j in 0 to 31 generate
1101     process(s32, B, fill132, is_left)
1102     begin
1103         if is_left = '0' then
1104             if j + 8 <= 31 then

```



```

1105         if B(3) = '1' then s32(2)(j) <= s32(1)(j+8);
1106         else s32(2)(j) <= s32(1)(j); end if;
1107     else
1108         if B(3) = '1' then s32(2)(j) <= fill132;
1109         else s32(2)(j) <= s32(1)(j); end if;
1110     end if;
1111 else
1112     if j - 8 >= 0 then
1113         if B(3) = '1' then s32(2)(j) <= s32(1)(j-8);
1114         else s32(2)(j) <= s32(1)(j); end if;
1115     else
1116         if B(3) = '1' then s32(2)(j) <= '0';
1117         else s32(2)(j) <= s32(1)(j); end if;
1118     end if;
1119 end if;
1120 end process;
1121 end generate;
1122
1123 gen_32_s2 : for j in 0 to 31 generate
1124     process(s32, B, fill132, is_left)
1125     begin
1126         if is_left = '0' then
1127             if j + 16 <= 31 then
1128                 if B(4) = '1' then s32(3)(j) <= s32(2)(j+16);
1129                 else s32(3)(j) <= s32(2)(j); end if;
1130             else
1131                 if B(4) = '1' then s32(3)(j) <= fill132;
1132                 else s32(3)(j) <= s32(2)(j); end if;
1133             end if;
1134         else
1135             if j - 16 >= 0 then
1136                 if B(4) = '1' then s32(3)(j) <= s32(2)(j-16);
1137                 else s32(3)(j) <= s32(2)(j); end if;
1138             else
1139                 if B(4) = '1' then s32(3)(j) <= '0';
1140                 else s32(3)(j) <= s32(2)(j); end if;
1141             end if;
1142         end if;
1143     end process;
1144 end generate;
1145
1146 Y_32ext <= (63 downto 32 => s32(3)(31)) & s32(3);
1147
1148
1149 process(s64, Y_32ext, A, ShiftFN, ExtWord)
1150 begin
1151     if ShiftFN = "00" then
1152         Y <= A;
1153     elsif ExtWord = '1' then
1154         Y <= Y_32ext;
1155     else
1156         Y <= s64(3);
1157     end if;
1158 end process;
1159
1160 End Architecture Dual824ChUni;
1161
1162
1163
1164 Architecture SingleSW2ChSep of ShiftUnit is
1165
1166     type Stage64 is array (0 to 6) of std_logic_vector(N-1 downto 0);
1167     signal sll64 : Stage64;
1168     signal srl64 : Stage64;
1169     signal sra64 : Stage64;
1170     signal A_right : std_logic_vector(N-1 downto 0);
1171     signal A_left : std_logic_vector(N-1 downto 0);
1172     signal shamt : std_logic_vector(5 downto 0);
1173

```

```

1174     signal Y_sll      : std_logic_vector(N-1 downto 0);
1175     signal Y_srl      : std_logic_vector(N-1 downto 0);
1176     signal Y_sra      : std_logic_vector(N-1 downto 0);
1177
1178 Begin
1179
1180
1181     process(A, ExtWord)
1182     begin
1183         if ExtWord = '1' then
1184             A_right(N-1 downto N/2) <= A(N/2-1 downto 0);
1185             A_right(N/2-1 downto 0) <= (others => '0');
1186         else
1187             A_right <= A;
1188         end if;
1189     end process;
1190
1191
1192     process(A, ExtWord)
1193     begin
1194         if ExtWord = '1' then
1195             A_left(N/2-1 downto 0) <= A(N/2-1 downto 0);
1196             A_left(N-1 downto N/2) <= (others => '0');
1197         else
1198             A_left <= A;
1199         end if;
1200     end process;
1201
1202
1203     shamt(4 downto 0) <= B(4 downto 0);
1204     shamt(5) <= B(5) when ExtWord = '0' else '0';
1205
1206
1207
1208     sll64(0) <= A_left;
1209     srl64(0) <= A_right;
1210     sra64(0) <= A_right;
1211
1212
1213     gen_sll : for i in 0 to 5 generate
1214         gen_sll_bits : for j in 0 to N-1 generate
1215             process(sll64, shamt)
1216                 variable shift_amt : integer := 2**i;
1217             begin
1218                 if j - shift_amt >= 0 then
1219                     if shamt(i) = '1' then
1220                         sll64(i+1)(j) <= sll64(i)(j - shift_amt);
1221                     else
1222                         sll64(i+1)(j) <= sll64(i)(j);
1223                     end if;
1224                 else
1225                     if shamt(i) = '1' then
1226                         sll64(i+1)(j) <= '0';
1227                     else
1228                         sll64(i+1)(j) <= sll64(i)(j);
1229                     end if;
1230                 end if;
1231             end process;
1232         end generate;
1233     end generate;
1234
1235
1236     gen_srl : for i in 0 to 5 generate
1237         gen_srl_bits : for j in 0 to N-1 generate
1238             process(srl64, shamt)
1239                 variable shift_amt : integer := 2**i;
1240             begin
1241                 if j + shift_amt <= N-1 then
1242                     if shamt(i) = '1' then

```

```

1243         srl64(i+1)(j) <= srl64(i)(j + shift_amt);
1244     else
1245         srl64(i+1)(j) <= srl64(i)(j);
1246     end if;
1247 else
1248     if shamt(i) = '1' then
1249         srl64(i+1)(j) <= '0';
1250     else
1251         srl64(i+1)(j) <= srl64(i)(j);
1252     end if;
1253 end if;
1254 end process;
1255 end generate;
1256 end generate;
1257
1258
1259 gen_sra : for i in 0 to 5 generate
1260     gen_sra_bits : for j in 0 to N-1 generate
1261         process(sra64, shamt, A, ExtWord)
1262             variable shift_amt : integer := 2**i;
1263             variable fill_bit : std_logic;
1264             begin
1265
1266                 fill_bit := A_right(N-1);
1267                 if j + shift_amt <= N-1 then
1268                     if shamt(i) = '1' then
1269                         sra64(i+1)(j) <= sra64(i)(j + shift_amt);
1270                     else
1271                         sra64(i+1)(j) <= sra64(i)(j);
1272                     end if;
1273                 else
1274                     if shamt(i) = '1' then
1275                         sra64(i+1)(j) <= fill_bit;
1276                     else
1277                         sra64(i+1)(j) <= sra64(i)(j);
1278                     end if;
1279                 end if;
1280             end process;
1281         end generate;
1282     end generate;
1283
1284     process(srl64, ExtWord)
1285     begin
1286         if ExtWord = '1' then
1287             Y_srl(N/2-1 downto 0) <= srl64(6)(N-1 downto N/2);
1288             Y_srl(N-1 downto N/2) <= (others => srl64(6)(N-1));
1289         else
1290             Y_srl <= srl64(6);
1291         end if;
1292     end process;
1293
1294     process(sra64, ExtWord)
1295     begin
1296         if ExtWord = '1' then
1297             Y_sra(N/2-1 downto 0) <= sra64(6)(N-1 downto N/2);
1298             Y_sra(N-1 downto N/2) <= (others => sra64(6)(N-1));
1299         else
1300             Y_sra <= sra64(6);
1301         end if;
1302     end process;
1303
1304
1305     process(sll64, ExtWord)
1306     begin
1307         if ExtWord = '1' then
1308             Y_sll(N/2-1 downto 0) <= sll64(6)(N/2-1 downto 0);
1309             Y_sll(N-1 downto N/2-1) <= (others => sll64(6)(N/2-1));
1310         else
1311             Y_sll <= sll64(6);

```

```

1312         end if;
1313     end process;
1314
1315
1316     with ShiftFN select Y <=
1317         Y_sll when "01",
1318         Y_srl when "10",
1319         Y_sra when "11",
1320         A     when others;
1321
1322 End Architecture SingleSW2ChSep;
1323
1324
1325 Architecture SingleZU2ChSep of ShiftUnit is
1326
1327     type Stage64 is array (0 to 6) of std_logic_vector(63 downto 0);
1328     signal sll64 : Stage64;
1329     signal srl64 : Stage64;
1330     signal sra64 : Stage64;
1331
1332     signal A_in      : std_logic_vector(63 downto 0);
1333     signal shamt     : std_logic_vector(5 downto 0);
1334     signal Y_sll     : std_logic_vector(63 downto 0);
1335     signal Y_srl     : std_logic_vector(63 downto 0);
1336     signal Y_sra     : std_logic_vector(63 downto 0);
1337
1338 Begin
1339
1340     A_in(31 downto 0) <= A(31 downto 0);
1341     A_in(63 downto 32) <= (others => '0') when ExtWord = '1'
1342         else A(63 downto 32);
1343
1344
1345     shamt(4 downto 0) <= B(4 downto 0);
1346     shamt(5)         <= B(5) when ExtWord = '0' else '0';
1347
1348
1349     sll64(0) <= A_in;
1350     srl64(0) <= A_in;
1351
1352
1353     process(A, ExtWord)
1354     begin
1355         if ExtWord = '1' then
1356             sra64(0)(31 downto 0) <= A(31 downto 0);
1357             sra64(0)(63 downto 32) <= (others => A(31));
1358         else
1359             sra64(0) <= A;
1360         end if;
1361     end process;
1362
1363
1364     gen_sll : for i in 0 to 5 generate
1365         gen_sll_bits : for j in 0 to 63 generate
1366             process(sll64, shamt)
1367                 variable shift_amt : integer := 2**i;
1368             begin
1369                 if j - shift_amt >= 0 then
1370                     if shamt(i) = '1' then
1371                         sll64(i+1)(j) <= sll64(i)(j - shift_amt);
1372                     else
1373                         sll64(i+1)(j) <= sll64(i)(j);
1374                     end if;
1375                 else
1376                     if shamt(i) = '1' then
1377                         sll64(i+1)(j) <= '0';
1378                     else
1379                         sll64(i+1)(j) <= sll64(i)(j);
1380                     end if;

```

```

1381         end if;
1382     end process;
1383 end generate;
1384 end generate;
1385
1386 gen_srl : for i in 0 to 5 generate
1387     gen_srl_bits : for j in 0 to 63 generate
1388         process(srl64, shamt)
1389             variable shift_amt : integer := 2**i;
1390         begin
1391             if j + shift_amt <= 63 then
1392                 if shamt(i) = '1' then
1393                     srl64(i+1)(j) <= srl64(i)(j + shift_amt);
1394                 else
1395                     srl64(i+1)(j) <= srl64(i)(j);
1396                 end if;
1397             else
1398                 if shamt(i) = '1' then
1399                     srl64(i+1)(j) <= '0';
1400                 else
1401                     srl64(i+1)(j) <= srl64(i)(j);
1402                 end if;
1403             end if;
1404         end process;
1405     end generate;
1406 end generate;
1407
1408
1409 gen_sra : for i in 0 to 5 generate
1410     gen_sra_bits : for j in 0 to 63 generate
1411         process(sra64, shamt, A, ExtWord)
1412             variable shift_amt : integer := 2**i;
1413             variable fill_bit : std_logic;
1414         begin
1415
1416             if ExtWord = '1' then fill_bit := A(31); else fill_bit := A(63); end if;
1417             if j + shift_amt <= 63 then
1418                 if shamt(i) = '1' then
1419                     sra64(i+1)(j) <= sra64(i)(j + shift_amt);
1420                 else
1421                     sra64(i+1)(j) <= sra64(i)(j);
1422                 end if;
1423             else
1424                 if shamt(i) = '1' then
1425                     sra64(i+1)(j) <= fill_bit;
1426                 else
1427                     sra64(i+1)(j) <= sra64(i)(j);
1428                 end if;
1429             end if;
1430         end process;
1431     end generate;
1432 end generate;
1433
1434
1435 process(sll64, ExtWord)
1436 begin
1437     if ExtWord = '1' then
1438         Y_sll(31 downto 0) <= sll64(6)(31 downto 0);
1439         Y_sll(63 downto 32) <= (others => sll64(6)(31));
1440     else
1441         Y_sll <= sll64(6);
1442     end if;
1443 end process;
1444
1445 process(srl64, ExtWord)
1446 begin
1447     if ExtWord = '1' then
1448         Y_srl(31 downto 0) <= srl64(6)(31 downto 0);
1449         Y_srl(63 downto 32) <= (others => srl64(6)(31));

```

```

1450         else
1451             Y_srl <= srl64(6);
1452         end if;
1453     end process;
1454
1455     process(sra64, ExtWord)
1456     begin
1457         if ExtWord = '1' then
1458             Y_sra(31 downto 0) <= sra64(6)(31 downto 0);
1459             Y_sra(63 downto 32) <= (others => sra64(6)(31));
1460         else
1461             Y_sra <= sra64(6);
1462         end if;
1463     end process;
1464
1465
1466     with ShiftFN select Y <=
1467         Y_sll    when "01",
1468         Y_srl    when "10",
1469         Y_sra    when "11",
1470         A        when others;
1471
1472     End Architecture SingleZU2ChSep;

```